

IMPERIAL

Deep Learning

Lecture 4: Practicalities and Convolutional Neural Networks

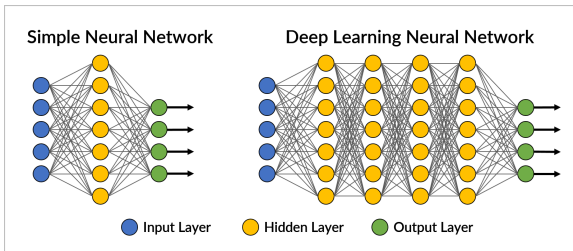
Pietro Ferraro

Table of Content

1. Recap
2. Training and Overfitting
3. Methods to reduce overfitting
4. Intuition about hierarchy
5. Convolutional Neural Networks

Recap

- We can use backpropagation to train a network and minimise a loss function.
- We have seen that we can build a ANN to solve regression using a linear output MSE as a loss function.
- We have seen that we can build a ANN to solve classification using a softmax output and cross-entropy as a loss function.



Practical comments

Roughly speaking we need to consider three things.

- Getting the data ready for learning.
- Selecting features and the model.
- Choosing an appropriate learning algorithm.

Getting the data ready

There are many steps in making a data set suitable for a machine learning algorithm.

As we have already discussed, at a very basic level, one must label a data set to make it suitable for machine learning algorithms. Often, this can be a very labour intensive process - think of the task of labelling images.

Sometimes, problems arise in which we must convert non-numeric labels in to numeric ones and vice-versa. Again, this is a task that must be undertaken with great care.

Practical aspects of Deep Learning

Getting the data ready

A more common data curation technique is called data normalisation. Normalisation is necessary to avoid having different scales in the training data. For example, suppose we postulate a model of the form:

$$y = ax_1 + bx_2 + c; \tag{1}$$

namely, the output depends on two input features x_1, x_2 . suppose further that x_1 is in the interval $[0.1, 0.7]$ and x_2 is in the interval $[-10000, 20000]$.

Situations like this almost always cause problems for machine learning algorithms. Often the lead to numeric problems, or just make the choice of learning parameters difficult.

Getting the data ready

Here the basic idea is to convert the actual range which a numerical technique can take to be in a standard interval; say $[0, 1]$. That is:

$$\tilde{x}(j) \leftarrow \frac{x(j) - \min_j x}{\max_j x - \min_j x} \quad (2)$$

or in the interval $[-1, 1]$ with zero mean,

$$\tilde{x}(j) \leftarrow \frac{x(j) - \mu}{\max_j x - \min_j x} \quad (3)$$

where μ is the average of the $x(i)$.

Getting the data ready

Another problem that often arises concerns data with missing features. This happens extremely often. Typical approaches to dealing with missing data include:

- removing examples with missing features;
- using special learning algorithms that can deal with missing features;
- using a **data imputation** technique (for example - replacing the missing feature by an average value of the feature in the training set).

Data and learning

Another issue concerns dividing the data into test and training sets. Remember, our objective is to learn. In fact, up to now in all our examples, we have not really considered learning. What we have done is to fit a sets of data-points to a parameterised model.

This is not learning. To learn, we wish our models to be able to generalise. That is:

make predictions on data that the model has not seen before.

Data and learning

Usually we divide our data into 3 parts.

- a training set;
- a test set;
- a validation set.

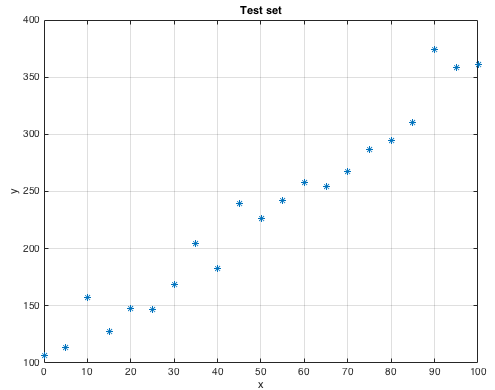
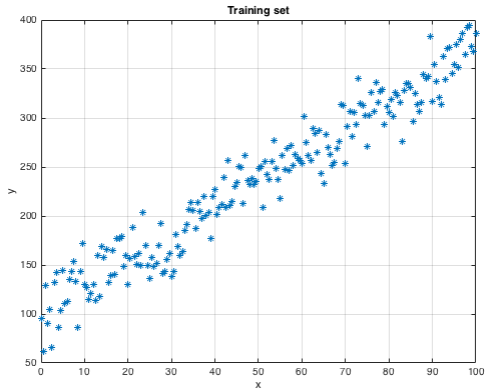
Sometimes, we use only a training set and a validation set.

Data and learning

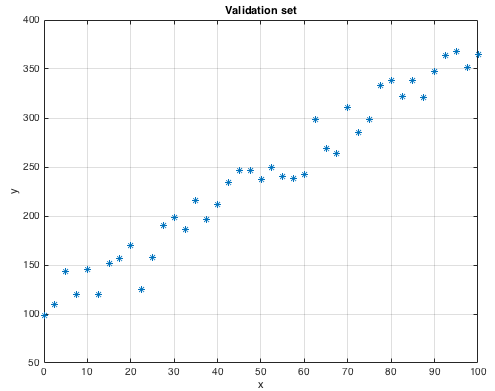
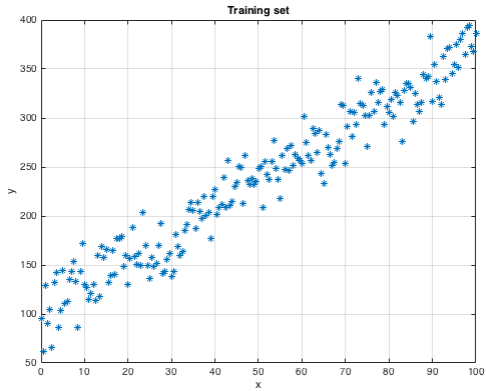
The meaning of these data sets is pretty self-explanatory.

Roughly speaking, the training set is used to find the models parameters (using for example, gradient descent); the validation set is to see how well the model has learned, and to detect over-fitting; and the test set is to provide an independent verification of the goodness of the model.

Data sets



Data sets



Model and feature selection

Consider a linear regression form. We postulated a model to fit to the data of the form:

$$y = mx + c. \quad (4)$$

Why did we pick a model of this form? How do we know that this model is more appropriate than, for example:

$$y = ax^2 + bx + c. \quad (5)$$

Isn't it a good idea to use richer model structures? The same thing can be said for ANNs. Why use a model with 10 neurons per layer and 2 layers when I can use 1000 neurons per layer and 500 layers?

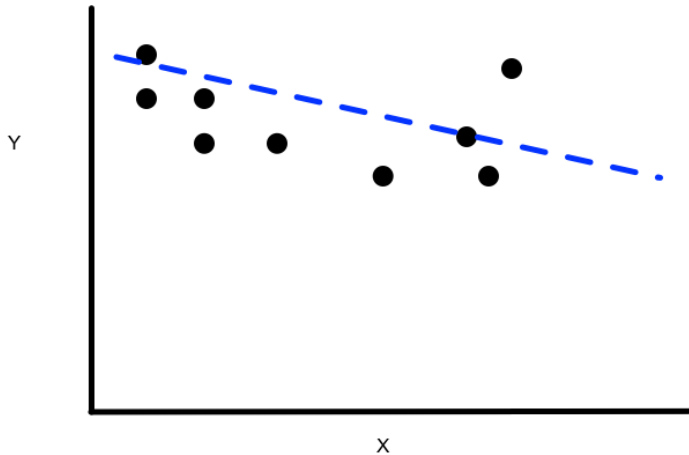
Model and feature selection

Isn't it a good idea to use richer model structures? Well, not always.

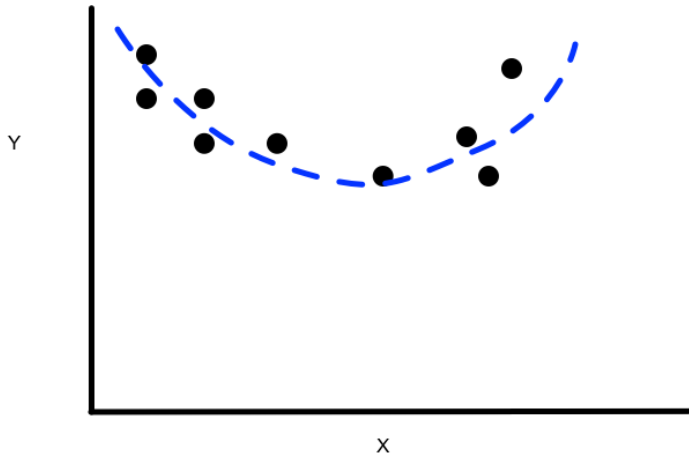
- Too few parameters → Don't really capture what is going on.
- Too many parameters → We end up fitting noise.

In reality, both situations are bad, and leads to models with very poor ability to generalise.

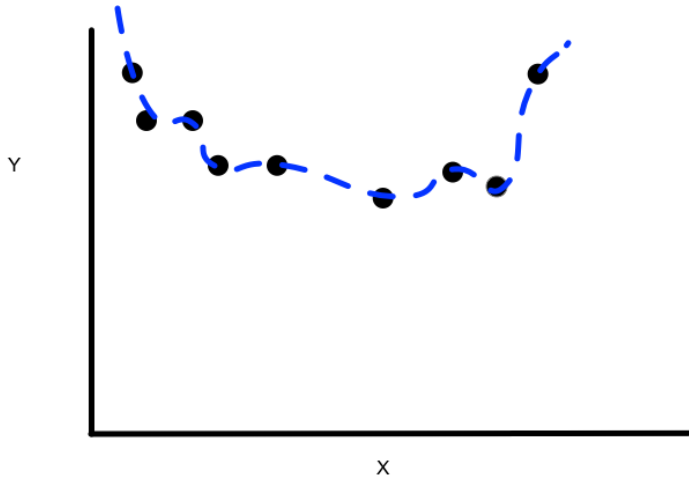
Model and feature selection



Model and feature selection



Model and feature selection



Overfitting and bias variance dilemma

In reality, selecting the structure for the model, and the features to predict labels, is usually based on prior information about the problem.

For example, in robotics, we often know the differential equation that describes the process.

Overfitted models usually have high variance. Change the data, and the model parameters change a lot. Underfitted models usually have a high bias. That is, they make a lot of errors in the test set. Striking a balance between the two is called the **bias-variance** tradeoff.

Regularisation

There are many solutions to the problem of overfitting. Perhaps the one that is most widely used is known as regularisation.

Regularisation is an umbrella term that constrains the model parameters: in other words, it forces the learning algorithm to find a less complex model.

Suppose, given a data-set, we postulate a neural network of the form: $y = \text{NN}(W, x)$, where W denotes the parameters of the model.

Regularisation

Up to now, we find the 'optimal', θ by finding the parameters that solves the following optimization:

$$\underset{W}{\text{minimize}} \quad J(W)$$

Regularisation involves adding another term to the optimization to constrain the parameters.

$$\underset{W}{\text{minimize}} \quad J(W) + \lambda g(W)$$

Regularisation

Typical choices of $g(W)$ are

$$g(W) = \sum_{ijk} w_{ijk}^2$$

$$g(W) = \sum_{ijk} |w_{ijk}|$$

The former is called L2 regularisation, or ridge regularisation. L2 regularisation looks for small parameter vectors.

The latter is called L1 regularisation, or LASSO regularisation. L1 regularisation looks for sparse parameter vectors.

Dropout

Another method to improve the effects of overfitting is to introduce dropout:

- Randomly sets the derivative of a fraction of neurons to zero during training.
- Prevents co-adaptation of neurons.
- Acts as an ensemble of models.

This is usually specified through a dropout rate. In practice, during training, we "deactivate" a portion of the neurons.

Parameter Tuning

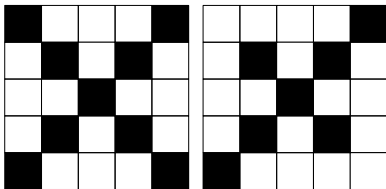
How do I tune the hyperparameter of the network? Main examples are: number of layers, number of neurons, learning rate η , dropout rate, regularization parameter λ , number of epochs, etc.

- Trial and error.
- Use already existing networks
- Use techniques like grid search or random search.

Convolutional Neural Networks

Images and patterns

A human would have no issues saying that these two images look very similar.



- Why?
- Can we use a Linear Neural Networks (LNNs) to classify them?

Why not use a LNN?

An image is just a matrix of numbers (actually, three or four matrices generally)

1	0	0	0	1
0				
0				
0				
1				

	0	0	0	0	1
	0				
	0				
	0				
	1				

		1	0	0	0	1
		0	1	0	1	0
		0	0	1	0	0
		0	1	0	1	0
		1	0	0	0	1

Why not use a LNN?

Let's consider a simple case with only one channel

1	0	0	0	1
0	1	0	1	0
0	0	1	0	0
0	1	0	1	0
1	0	0	0	1

0	0	0	0	1
0	1	0	1	0
0	0	1	0	0
0	1	0	1	0
1	0	0	0	0

Why not use a LNN?

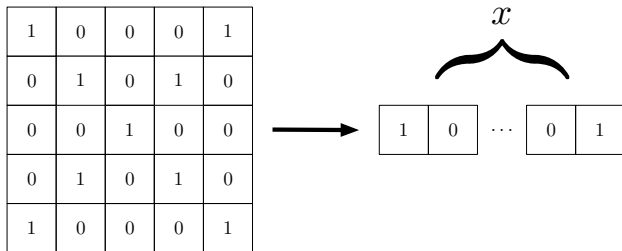
- Can you think of a disadvantage in using a LNN when analysing an image?

Why not use a LNN?

- Can you think of a disadvantage in using a LNN when analysing an image?
- What are we losing (or rather, what are we getting too much of), in terms of information?

Why not use a LNN?

- Can you think of a disadvantage in using a LNN when analysing an image (try to count the number of weights needed for a 1024×1024 image in a LNN with 64 neurons as a first layer)?
- What are we losing, in terms of information?



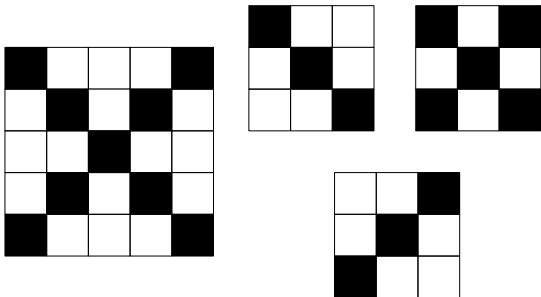
Features

Images and data, in general, often present Spatial Features and the order in which data are presented matters. In a LNN, since each neuron looks at the whole input, we lose entirely the spatial information

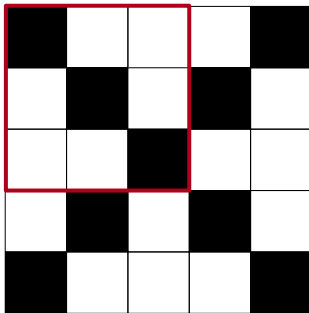
- Edges, curves
- Images are often composed in a hierarchical way
- How can we include them in our Neural Network?

Convolutional Kernels

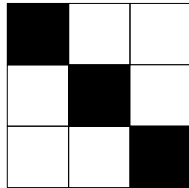
A good way of spotting features doing it is by employing the convolutional operator and a set of "filters" or "Convolutional Kernels"



Convolutional Kernels - Example



$$X = \begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix}$$



$$W_1$$

$$W_1 * X_{ij}$$

Convolutional Kernels - Example

Let's use -1 and 1, instead of 0 and 1, to make the difference between patches more striking

1	-1	-1	-1	1
-1	1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	1	-1
1	-1	-1	-1	1

 $*$

1	-1	-1
-1	1	-1
-1	-1	1

 $=$

9		

$$(1 \cdot 1) + (-1 \cdot -1) + (-1 \cdot -1) + (-1 \cdot -1) + (1 \cdot 1) + (-1 \cdot -1) + (-1 \cdot -1) + (-1 \cdot -1) + (1 \cdot 1) = 9$$

Convolutional Kernels - Example

In this portion of the image the activation is much lower

1	-1	-1	-1	1
-1	1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	1	-1
1	-1	-1	-1	1

 $*$

1	-1	-1
-1	1	-1
-1	-1	1

 $=$

9	-3	

$$(-1 \cdot 1) + (-1 \cdot -1) + (-1 \cdot -1) + (1 \cdot -1) + (-1 \cdot 1) + (1 \cdot -1) + (-1 \cdot -1) + (1 \cdot -1) + (-1 \cdot 1) = -3$$

Convolutional Kernels - Example

If the Kernel "finds" the feature in a patch, the corresponding output will be a large value

1	-1	-1	-1	1
-1	1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	1	-1
1	-1	-1	-1	1

 $*$

1	-1	-1
-1	1	-1
-1	-1	1

 $=$

9	-3	1
-3	3	-3
1	-3	9

Convolutional Kernels - Example

If the Kernel "finds" the feature in a patch, the corresponding output will be a large value

1	-1	-1	-1	1
-1	1	-1	1	-1
-1	-1	1	-1	-1
-1	1	-1	1	-1
1	-1	-1	-1	1

 $*$

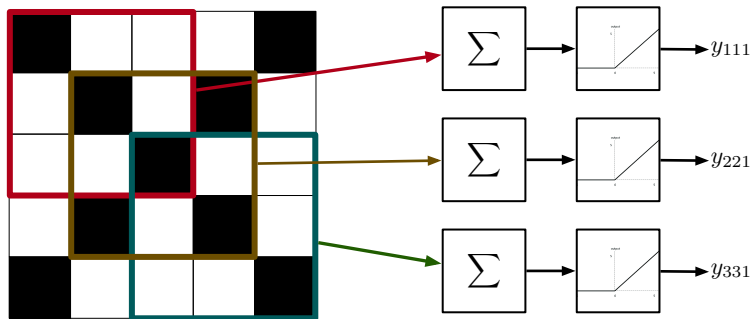
1	-1	-1
-1	1	-1
-1	-1	1

 $=$

9	-3	1
-3	3	-3
1	-3	9

Convolutional Layers

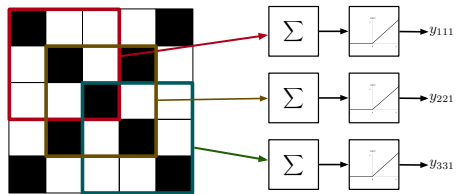
We talked about kernels and features. How is this connected to neurons? (Showing only three neurons out of nine)



Convolutional Layers

Each neuron is connected to a limited amount of inputs, called the "receptive field" of that specific neuron. For all these neurons the weights (the kernel) are the same!

$$h(s) = \begin{cases} s & \text{if } s \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

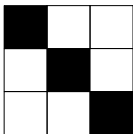


$$W_1 = \begin{bmatrix} w_{111} & w_{121} & w_{131} \\ w_{211} & w_{221} & w_{231} \\ w_{311} & w_{321} & w_{331} \end{bmatrix}$$

$$y_{ij1} = h(W_1 * X_{ij} + b_1)$$

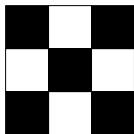
Convolutional Layers

For our specific example, since we are considering three kernels, we end up having three "groups of neurons"



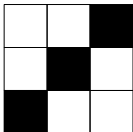
$$W_1 = \begin{bmatrix} w_{111} & w_{121} & w_{131} \\ w_{211} & w_{221} & w_{231} \\ w_{311} & w_{321} & w_{331} \end{bmatrix}$$

$$y_{ij1} = h(W_1 * X_{ij} + b_1)$$



$$W_2 = \begin{bmatrix} w_{112} & w_{122} & w_{132} \\ w_{212} & w_{222} & w_{232} \\ w_{312} & w_{322} & w_{332} \end{bmatrix}$$

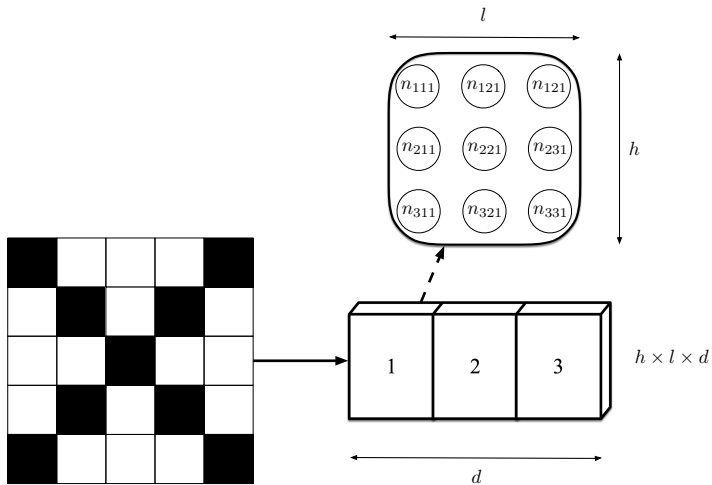
$$y_{ij2} = h(W_2 * X_{ij} + b_2)$$



$$W_3 = \begin{bmatrix} w_{113} & w_{123} & w_{133} \\ w_{213} & w_{223} & w_{233} \\ w_{313} & w_{323} & w_{333} \end{bmatrix}$$

$$y_{ij3} = h(W_3 * X_{ij} + b_3)$$

Convolutional Layers

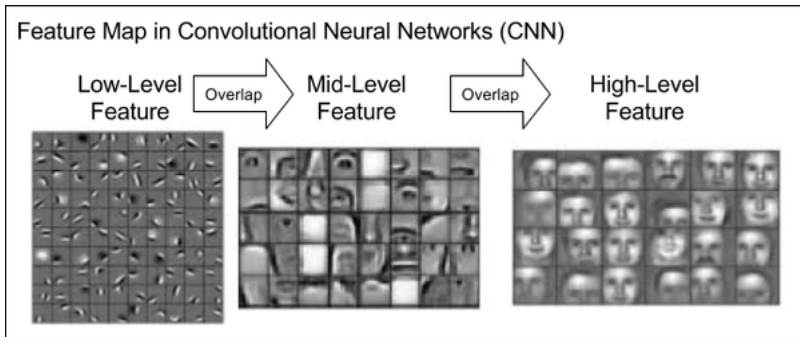


Convolutional Layers

So, a convolutional layer is composed of $h \times l \times d$ neurons and the resulting output is a $h \times l \times d$ array (tensor) that is then fed to the next layer. The set of neurons that share the same feature (the ones shown in the previous slide) are usually called a "feature map".

What do you think will the next layer do? Are the features of the next layer related to the one before?

Convolutional Layers



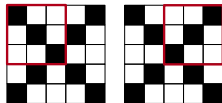
A bit more on Convolutional Kernels...

You will have noticed that the resulting output, in our example, is smaller than the original image. This depends on a series of hyperparameters:

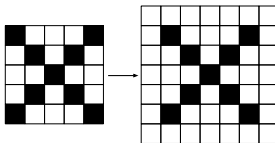
- Kernel dimensions ($k_{xi} \times k_{yi}$): Dimension of the matrices W_i (it's normally square but nothing prevents $k_{xi} \neq k_{yi}$);
- Stride (S): the distance between the center of the current and the next convolution operation. In our example we set the stride to 1;
- Padding (P): Filling the image with a series of 0 on the edges in order to avoid cropping effects due to kernels.

Stride and Padding example

Stride, $S = 3$



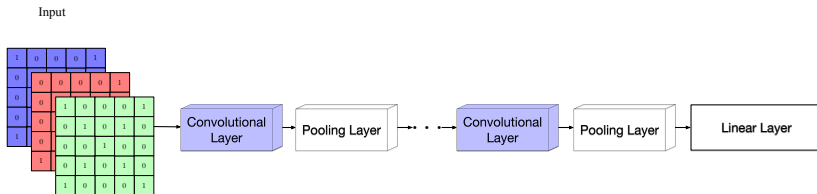
Padding, $P = 1$



CNN Architecture

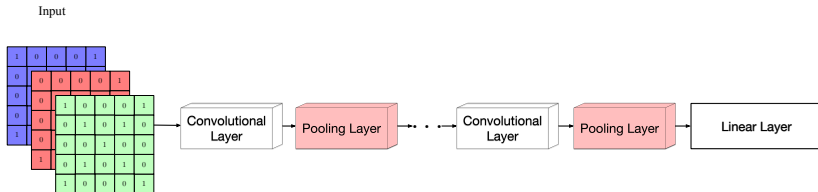
Let's now take a look at a complete convolutional network to classify images.

- Convolutional Layer is the one just explained
- Let's take a look at the other two now

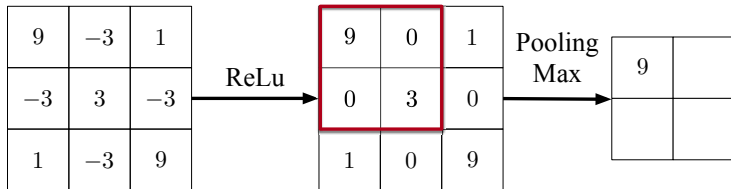


Pooling Layer

This layer is used to reduce the dimension of the input and to "focus" on larger areas of the image (otherwise to capture combination of features the dimension of the kernel in successive layers should keep increasing)

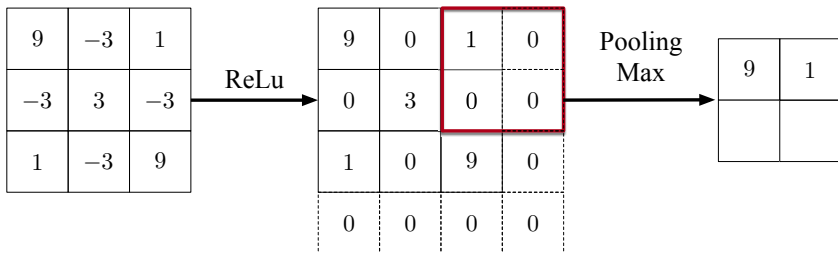


Pooling Layer Example



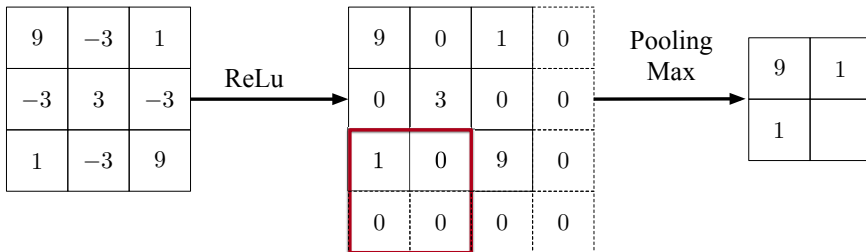
Pooling Layer Example

Since in this example the dimensions of the matrix are odd, we add two lines of zeros (notice that the result could be different, depending on where you do it, but for very large images this wouldn't really matter).



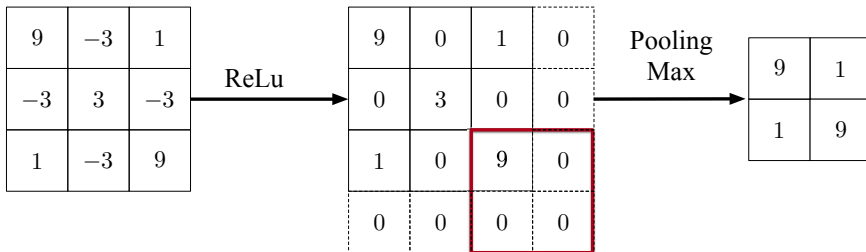
Pooling Layer Example

Since in this example the dimensions of the matrix are odd, we add two lines of zeros (notice that the result could be different, depending on where you do it, but for very large images this wouldn't really matter).



Pooling Layer Example

Since in this example the dimensions of the matrix are odd, we add two lines of zeros (notice that the result could be different, depending on where you do it, but for very large images this wouldn't really matter).



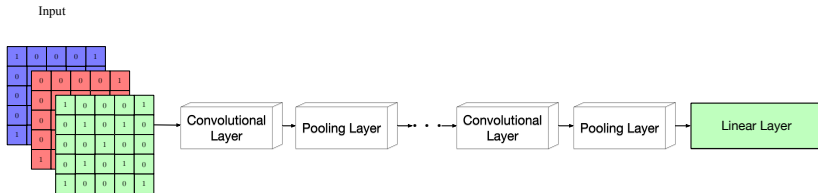
Pooling Layer

Pooling Layers can be of different kinds

- MaxPool: We take the maximum value in the pooling filter
- AvgPool: We average over the values in the pooling filter
- The dimension of the pooling filter is a design choice
- The stride of the pooling filter is almost always equal to its x-size

Linear Layer

Finally, the Linear Layer combines the outputs of the last Convolutional layer and uses a classifier structure to provide a probability distribution for our input image



Linear Layer

Do you remember the last exercise in Lab 2? We used the softmax function and the cross-entropy loss to find a probability distribution for our inputs.

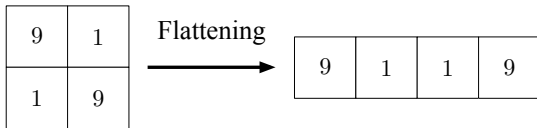
$$y_i = f_i(s) = \frac{e_i^s}{\sum_{j=1}^N e^{s_j}} \quad (6)$$

$$H = - \sum_{i=1}^N z_i \log y_i \quad (7)$$

With $z = [z_1, \dots, z_N]$ being a one-hot vector and $\sum_{i=1}^N y_i = 1$

Linear Layer

Of course, one thing to remember is that the output of the last pooling layer is a series of matrices (3 matrices in our example). These need to be "flattened" so that they can be used as an input for our last linear layer.



In our specific example, we will flatten three matrices and concatenate the resulting vectors, ending up with a 12-dimensional input.

Learning

At this point everything is set and all we need to do is initialize our weights, setting up our training set, setting up the labels and then use the backpropagation algorithm to teach our network to recognise the desired outputs.

$$\mathbf{w}_{ij}^{\text{lin}} \leftarrow \mathbf{w}_{ij}^{\text{lin}} - \alpha \frac{\partial H}{\partial \mathbf{w}_{ij}^{\text{lin}}} \quad (8)$$

$$\mathbf{w}_{ijz}^{\text{conv}} \leftarrow \mathbf{w}_{ijz}^{\text{conv}} - \alpha \frac{\partial H}{\partial \mathbf{w}_{ijz}^{\text{conv}}} \quad (9)$$

Notice that equation (9) is how the network learns the features that we have considered given, up to this point (remember also that neurons in the convolutional layer share most of the weights).

Any questions?